

## A Beginner Guide To Automating Code With GitHub Actions

GitHub Actions helps you automate your software build, test, and deployment pipeline directly inside your repository. Learning this tool saves you hours of manual work and ensures your code works every single time you push.

### What you'll learn:

1. Creating Your First Workflow File
2. Pulling Your Code into the Runner
3. Setting Up Your Runtime Environment
4. Storing Secrets and Sensitive Keys Safely

#### Aman Raj Singh

Cloud Operations Engineer @ iManage

[linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

# 1

TIP 1 OF 4

## 1. Creating Your First Workflow File

Workflows live in a special folder named '.github/workflows' at the root of your project. They are written in YAML format, which uses spaces to organize instructions step by step. When you create this file and push it, GitHub immediately recognizes it as an automated pipeline.

```
$name: Simple CI
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello from GitHub Actions!"
```

# Cloud & DevOps Daily

GitHub Actions: Automate Your Workflow for Free

Monday, August 24, 2026

Swipe to tip 2 >> [linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

## 2

TIP 2 OF 4

## 2. Pulling Your Code into the Runner

GitHub runs your workflow on a fresh cloud machine (called a runner). By default, this virtual machine starts completely empty and does not have your project files. You must use the official 'checkout' action to download your code into the machine before running any tests.

```
$jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Fetch repository code
        uses: actions/checkout@v4
```

# Cloud & DevOps Daily

GitHub Actions: Automate Your Workflow for Free

Monday, August 24, 2026

Swipe to tip 3 >> [linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

## 3

TIP 3 OF 4

### 3. Setting Up Your Runtime Environment

Before running tests or building software, you need the right runtime installed, like Python or Node.js. Pre-built actions allow you to set up your desired language version in seconds without typing complex installation commands.

#### `$steps:`

- `uses: actions/checkout@v4`
- `name: Set up Node.js`  
  `uses: actions/setup-node@v4`  
  `with:`  
    `node-version: '20'`
- `run: npm install`
- `run: npm test`

# Cloud & DevOps Daily

GitHub Actions: Automate Your Workflow for Free

Monday, August 24, 2026

Swipe to tip 4 >> [linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

## 4

TIP 4 OF 4

### 4. Storing Secrets and Sensitive Keys Safely

Never paste API keys or database passwords inside your code files. Go to your GitHub Repository > Settings > Secrets and variables > Actions, and save them there securely. You can then reference these secrets in your workflow YAML file safely.

```
$steps:
  - name: Connect to Service
    env:
      API_KEY: ${ secrets.MY_API_KEY }
    run: |
      echo "Connecting using safe environment key..."
```



# Cloud & DevOps Daily

GitHub Actions: Automate Your Workflow for Free

Monday, August 24, 2026

See Key Takeaways on next page >> [linkedin.com/in/mramanrajsingh](https://linkedin.com/in/mramanrajsingh)

## Key Takeaways

- + Always specify action versions explicitly (e.g., uses: actions/checkout@v4)
- + Keep workflow files small and split complex tasks into multiple jobs
- + Use 'on: pull\_request' to test incoming code before merging it to main
- + Check the Actions tab on GitHub to read log outputs whenever a build fails
- + Store all API tokens and access keys in GitHub Secrets, never in plain text

## Watch Out For

- ! Forgetting that YAML files are sensitive to indentation and spaces
- ! Trying to run code without using 'actions/checkout@v4' first
- ! Hardcoding secret passwords directly into public repository workflow files

**Follow for daily Cloud & DevOps guides!**

[linkedin.com/in/mrmanrajsingh](https://linkedin.com/in/mrmanrajsingh)